

Enterprise AI Use-Case Programme

Seventy-five use cases in three years — designing a programme that survives contact with delivery

Most enterprise AI programmes are scoped as a use-case count and a timeline. That is not a design. This is the arithmetic, the wave structure and the governance that decide whether the number is achievable.

~75	27	5	4	3	150–200
use cases to deliver	published as indicative	banking domains	delivery waves	showcase use cases	staff to train

PROGRAMME PARAMETERS · PUBLISHED IN THE TENDER SCOPE OF WORK

Production-grade, not pilots · co-delivered with bank teams · independent QA at one per three developers · full capability transfer in three years

Aditya Singh · Programme-design analysis authored for solution and bid strategy · 2026

Seventy-five use cases is a procurement number — the delivery question is the mix, the wave structure and who is capable of running it at the end

Situation

A three-year programme to deliver roughly seventy-five GenAI, agentic and machine-learning use cases across five banking domains, on a platform being built in parallel.

Twenty-seven are published as indicative. The rest will be identified by the bank in waves.

Complication

Three things make the number harder than it looks. The use cases are not equivalent — an enterprise RAG build and a policy-search assistant differ by an order of magnitude. The delivery window is shorter than the volume implies. And they must be co-delivered, which is slower than delivering them yourself.

A programme priced on an average use case is priced on a use case that does not exist.

The design

Size by complexity tier, not by count. Structure the waves so the platform is proven by the first, and the bank's own teams are leading by the third.

Then make capability transfer a delivery constraint with its own acceptance criteria — not a training line item at the end.

THE NUMBERS THAT MATTER

~75

use cases in scope

3

complexity tiers

4

waves over 3 years

1:3

QA to developer ratio

3

showcase demonstrations

100%

training completion, twice

The central claim: a use-case programme fails on mix and transfer, not on technology. Sizing by count produces a plan that is wrong on day one; sizing by complexity tier and designing the exit from the start produces one that survives.

Programme parameters are published; the sizing models are explicitly illustrative and exist to show the method rather than to state a result

TENDER DOCUMENTS Published context Scope, volumes, domains and delivery obligations as published in the tender and pre-bid minutes. • ~75 use cases over three years • 27 indicative, across 5 domains • 3 named showcase use cases • Wave structure and milestones • QA ratio and red-team cadence • Training and transfer obligations	PROGRAMME DESIGN My contribution The complexity tiering, wave design, resourcing arithmetic and transfer-as-constraint argument. • Three-tier complexity model • Wave sequencing logic • Resourcing arithmetic and the gap • Capability-transfer design • Governance and acceptance model • Applied to live enterprise bids	ILLUSTRATIVE Modelled Effort, mix and throughput figures used to demonstrate the arithmetic. Directional, not measured. • Effort per complexity tier • Mix assumption across 75 • Throughput and pod maths • Wave capacity curve • Replace with programme actuals • Method transfers unchanged	OUT OF SCOPE Not claimed Excluded deliberately. This is a design method, not a bid response and not a delivery record. • Any employer's bid position • Delivery of this programme • Any pricing or rate assumption • Any confidential assessment • The tender outcome • Any client-identifying detail
---	---	--	---

Twenty-seven indicative use cases across five domains — and the domains differ far more in delivery difficulty than the count suggests

Credit lifecycle 6 indicative

Underwriting support
Risk assessment
Credit scoring
Collections optimisation
Early-warning signals
Decision support

Risk & compliance 6 indicative

Risk monitoring
Regulatory reporting
Anti-money-laundering
Know-your-customer
Fraud and anomaly
Surveillance

Banking operations 5 indicative

Service automation
Document processing
Workflow automation
Exception management
Process optimisation

Technology & IT ops 5 indicative

Code assistants
IT operations support
Knowledge operations
Incident analysis
Service automation

HR & enterprise 5 indicative

HR assistant
Policy search
Procurement intelligence
Employee knowledge
Admin automation

Where the difficulty concentrates

- Credit and risk use cases touch regulated decisions, so every one carries model-risk governance, explainability and audit obligations.
- They also need the most sensitive data, which means the longest access approvals.
- Twelve of twenty-seven sit in these two domains — nearly half the published portfolio.

Where it is genuinely lighter

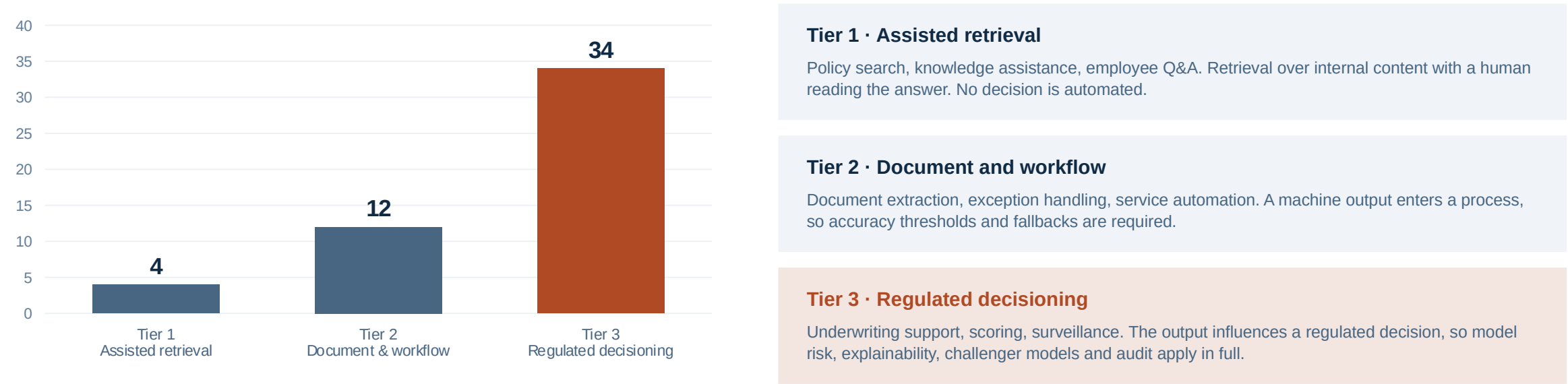
- HR, enterprise and IT-operations use cases are mostly retrieval and assistance over internal content.
- They carry real value and low regulatory weight, and they are where a programme can build velocity early.
- Ten of twenty-seven — and they should not be sequenced last.

The sequencing implication

- Lead with the lighter domains to prove the platform, the pipeline and the governance model in production.
- Use that proof to earn the data access the credit and risk use cases need.
- Sequencing the hardest domain first is a common and expensive instinct — it stalls the whole programme at the access-approval stage.

SIZING

Three complexity tiers differ by roughly an order of magnitude — which is why sizing by use-case count produces a plan that is wrong on day one



Illustrative effort by tier — the ratio between tiers is the argument, not the absolute values

The arithmetic that decides the programme

Take seventy-five use cases at a plausible mix — say forty per cent tier one, forty per cent tier two and twenty per cent tier three. That is roughly 120 plus 360 plus 510 person-weeks, near a thousand in total, before co-delivery overhead, independent QA at one per three developers, and red-team cycles.

The same seventy-five sized at an average of ten person-weeks comes to 750 — a third less, and the shortfall lands entirely in the tier-three use cases, which are the regulated ones the bank cares most about. Sizing by count does not produce a slightly optimistic plan. It produces a plan that fails precisely where failure is most visible.

THE SHOWCASE THREE

Three named use cases must be demonstrated by every bidder — and each one tests a different capability the rest of the programme depends on

GENERATIVE AI

Enterprise retrieval over tens of thousands of pages

Retrieval-augmented generation across a very large internal corpus, answering with citation and refusing when the corpus does not support an answer.

WHAT IT ACTUALLY TESTS

- Corpus preparation and chunking at scale
- Retrieval quality, not model quality
- Grounding, citation and refusal behaviour
- Latency at enterprise document volume

Tests whether the platform can be trusted with unstructured internal knowledge. Almost every tier-one use case in the programme is a variant of this.

AGENTIC AI

Scanned-document processing with government-API verification

An agent reads scanned identity and entity documents, extracts structured fields, and verifies them against external government interfaces end to end.

WHAT IT ACTUALLY TESTS

- Extraction accuracy on poor-quality scans
- Agentic orchestration across external calls
- Failure handling when a source is unavailable
- Auditability of an autonomous sequence

Tests whether agents can act, not just answer. This is the capability that separates an assistant platform from an automation platform.

MACHINE LEARNING

Real-time cash-flow underwriting from consented financial data

Cash-flow-based credit assessment built on consented account-aggregator and tax-return data, produced in real time at the point of decision.

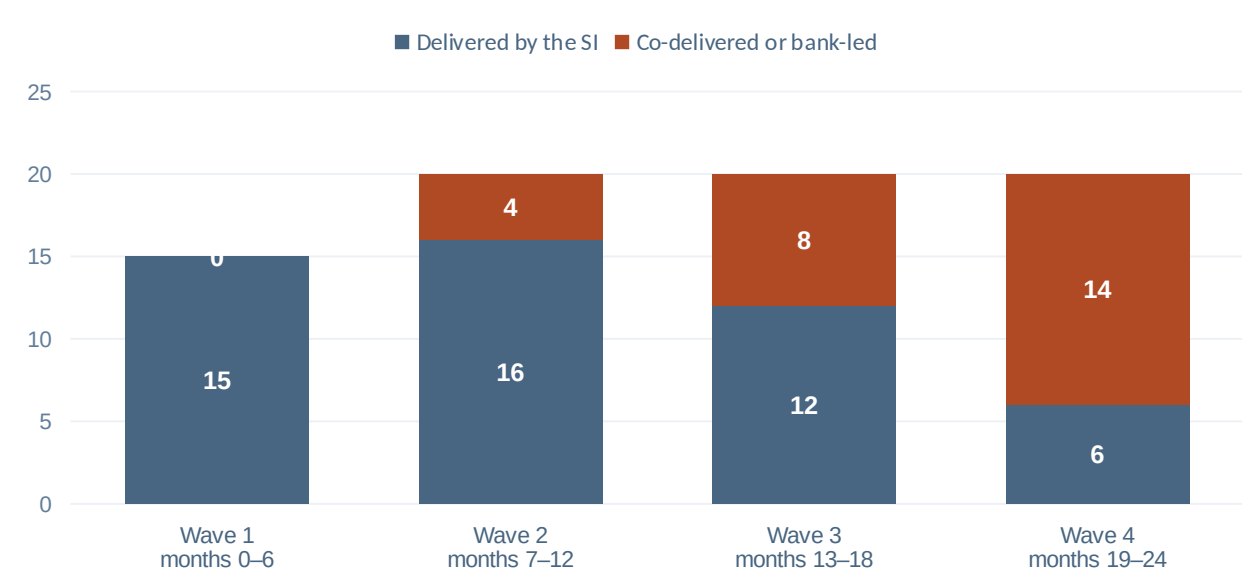
WHAT IT ACTUALLY TESTS

- Consented-data ingestion and normalisation
- Feature engineering on messy real inputs
- Model risk, explainability and challengers
- Real-time inference inside a credit flow

Tests whether the platform can carry a regulated decision. This is the hardest tier-three pattern and the one with the clearest commercial value.

The three are not a sample of the programme. They are a test of the three capability classes every remaining use case will draw on.

Four waves, but the capacity curve is not flat — the first wave carries platform proof and the last carries the bank running it alone



Illustrative wave design — the shift in who delivers is the design, not the volume

Wave 1 — prove the platform

Lighter tiers only. Establish the pipeline, the governance path and the acceptance model in production.

Wave 2 — introduce complexity

First regulated use cases. Bank engineers pair on delivery rather than observe it.

Wave 3 — invert the ratio

Bank teams lead, the SI reviews. This is the wave where transfer either happens or visibly has not.

Wave 4 — withdraw

The SI delivers only the hardest remainder. If the bank cannot carry the rest here, the exit criteria were never real.

Why the delivery split matters more than the volume split

Every programme of this shape publishes a volume curve. Almost none publishes an ownership curve. But the contract's actual end-state obligation is that the bank can build, deploy, govern and run use cases independently — which is a statement about wave four, not about the total. Designing the ownership shift explicitly, wave by wave, with acceptance criteria attached, is the only way that obligation is met on time rather than discovered at the handover.

The published window and the published volume do not reconcile at any credible effort assumption — which makes it the first question to ask, not the last

Assumption	Optimistic	Plausible	Regulated-heavy	Unit
Average effort per use case	8	13	19	person-weeks
Total delivery effort	600	975	1,425	person-weeks
Co-delivery overhead at 25%	150	244	356	person-weeks
Independent QA at one per three	250	406	594	person-weeks
Total programme effort	1,000	1,625	2,375	person-weeks
Sustained team required	16	26	38	people, 3 years

1,625

person-weeks at a plausible mix

26

people sustained for three years

2.7×

the optimistic estimate at the top end

1

question that resolves it

The twofold spread is driven almost entirely by the assumed complexity mix, which the tender does not state. Asking costs one clarification question before the deadline; guessing costs the difference between a viable programme and a loss-making one for three years.

Four obligations sit outside the use-case count and are routinely under-priced — together they are a substantial share of programme cost

Independent quality assurance

A service-assurance team structurally separate from delivery, at a minimum of one QA per three developers, owning SLAs, audits and continuous improvement.

Roughly a quarter of delivery effort again — and it cannot be absorbed into the delivery pods, because independence is the requirement.

Standing AI red team

Adversarial exercises at least half-yearly against prompt injection, jailbreaks, agent misuse and data leakage, at critical-infrastructure security standard.

A specialist capability that must exist continuously, not a penetration test bought once at go-live.

AI control tower

Use-case portfolio monitoring, model inventory and lifecycle, drift and performance, risk and compliance oversight, cost and consumption management.

A distinct product deliverable with its own build and run cost, frequently mistaken for a dashboard.

Capability transfer

Two full training cycles — platform, then use-case delivery — with SOPs, runbooks, documentation and an end state where bank teams operate independently.

Priced as a training line, delivered as a two-year constraint on how every use case is built.

None of the four appears in the use-case count, and all four run for the full term. A bid that prices seventy-five use cases and treats these as overhead has under-priced the programme by a margin that no delivery efficiency can recover.

Four rules for designing a large AI use-case programme

- | | | |
|----|--|---|
| 01 | Size by tier, never by count | Use cases differ by close to an order of magnitude in effort. An average is a number that describes no actual use case, and the shortfall always lands on the regulated ones. |
| 02 | Design the ownership curve, not just the volume curve | Every programme publishes how many use cases land per wave. The contract's real end state is who is delivering them by the final wave — design that shift explicitly, with acceptance criteria. |
| 03 | Sequence for access, not for value | The highest-value use cases usually need the most sensitive data and the longest approvals. Lead with lighter domains to prove the platform, then use that proof to earn the access. |
| 04 | Price the obligations outside the count | Independent QA, a standing red team, the control tower and two training cycles run for the full term and appear nowhere in the use-case number. They are a large share of the cost. |

Seventy-five is a procurement number. The mix, the ownership curve and the obligations outside the count are what determine whether it is a plan or a wish.